NYPkaki

How It Works

A technical walkthrough of every feature and the reasoning behind how it was built — for explaining the project to your teacher.

WHAT I USED

Frontend	React 19 + Vite, hand-written CSS (no UI library)	**Routing**	react-router-dom
Auth	Firebase Authentication (email/password)	**Database**	Cloud Firestore (NoSQL, client SDK only)
AI	Firebase AI Logic → Google Gemini (gemini-flash-latest)	**Hosting**	Vercel
Security	Firestore Security Rules	**Testing**	Playwright (automated browser testing)

Architecture, in one line: there is no custom backend server. Firebase *is* the backend — Authentication handles login, Firestore is the database, and Firebase AI Logic calls Gemini directly from the browser. What would normally be server-side authorization logic is instead enforced by **Firestore Security Rules** — a rules file Firebase evaluates on every read/write, so a student genuinely cannot read or write data they shouldn't, even though there's no server checking that in the middle. This is a standard "Backend-as-a-Service" pattern, not a shortcut — it's the same category of architecture real products (Firebase-backed apps, Supabase apps) use in production.

FEATURE BY FEATURE

01 Sign-up & login

What: Register with an NYP admin number; name and course auto-fill if the admin has pre-imported the official roster.

How: Firebase Authentication creates the email/password account; a matching `users` document in Firestore stores the profile (name, course, year, bio). If the admin number matches a row in the pre-imported `studentDirectory` collection, those fields auto-fill during registration.

02 The matching engine

What: Ranks every possible tutor for a request with a 0–100% compatibility score.

How: A deterministic, weighted formula — not random, not AI — adds up six factors: **module match (35 pts)** — exact competency match scores full, a related module (per NYP's own course groupings) scores half; **topic match (20 pts)** — proportion of requested topics the tutor actually lists, nudged by their self-rated confidence; **availability overlap (20 pts)** — real overlapping free time between both saved weekly schedules; **format match (10 pts)** — online/in-person preference; **tutor experience (5 pts)** — times they've tutored this exact module before; **tutor reliability (10 pts)** — real average rating + response rate from actual completed sessions, with new tutors scored at full marks so no history is never treated as a black mark.

Why: a percentage a student is about to act on has to be explainable and reproducible — same inputs, same score, every time, and every point traceable to a real reason.

03 "Why this match?"

What: A breakdown modal explaining a specific score.

How: Reuses the exact same six factors from the matching engine, translated into a plain checklist — ✓ for a real positive, ✗ for a real shortfall, a neutral dash for "no history yet" — plus the single longest real overlapping availability slot as the suggested meeting time.

04 Natural-language request ("Describe what you need")

What: Type a sentence like "*I need help with recursion for my test on Friday, I'm free Wednesday after 5pm*" and it extracts module, topic, goal, deadline, urgency, availability, and format.

How: Split on purpose. AI (Gemini) only does the two things a model is genuinely needed for: picking which competency the text is about, and writing a one-line learning goal. Everything else — urgency, the deadline's weekday math, "Wednesday after 5pm" turning into an actual day + time window, format preference — is parsed with plain deterministic string matching, no AI involved. If the AI call fails or hits the free tier's rate limit, it silently falls back to the local parser.

Why: a date or a percentage should never depend on a model "getting it right." Only the genuinely language-shaped part (summarizing intent) uses one.

05 AI session plans

What: Once a session is scheduled, generates a structured plan: Warm-up & Review, Main Concepts, Practice Activity, Questions, Final Recap.

How: AI writes the content for each of five *fixed* stages using the student's real module/topics/goal as context. The time split across those stages is plain arithmetic (fixed percentages of the session length), not model output, so the numbers always add up correctly. Falls back to a template (still built from the real topics) if AI is unavailable. Tutor can edit everything before the session.

06 Tutor reliability & feedback

What: Rating, sessions-completed count, and response rate shown on every tutor card.

How: After a session is marked complete, the student leaves a 1–5 star rating + helpful flag + optional comment (a `feedback` document, one per person per session). A tutor's displayed stats are computed live by aggregating real `feedback`, `sessions`, and `matchRequests` documents every time their card renders — there is no stored "rating" number anywhere that could drift out of sync with reality.

07 Requests → Sessions lifecycle

What: Send a request → tutor accepts/declines → arrange a real day/time/format/location → shows as Upcoming → mark Completed → leave feedback.

How: Each stage is a status field on Firestore documents (`matchRequests`, `sessions`), and Security Rules restrict reads/writes to only the two people actually involved in that match — the rules are the authorization logic, there's no server checking this separately.

08 Admin roster import / export

What: Bulk-import the official student roster from a CSV so registration auto-fills by admin number; export the registered-student list back to CSV.

How: A small, dependency-free CSV/TSV parser — deliberately *not* using the popular xlsx/SheetJS library, which has known unpatched vulnerabilities — reads "Admin No" / "Name" / "Course" / "Year" columns (recognizing a few header spellings) and writes each row as a Firestore document keyed by admin number. Export just re-serializes the currently-loaded list to a CSV file entirely in the browser, no server round-trip.

09 Notifications

What: A bell with unread badges for new requests, accepted tutoring, and scheduled classes.

How: Computed on the fly from data that already exists (`matchRequests` , `classRequests` , `classInterests`) rather than a dedicated notifications table — zero new backend work to add the feature. "Seen" state lives in the browser's local storage per device, since there's no server to track it centrally.

10 Availability calendar

What: A weekly drag-select grid for marking free time.

How: Each 30-minute cell is a pointer-driven toggle; dragging selects a contiguous range in one motion. Saved as individual `availability` documents per user — this exact data is what powers the "availability overlap" score factor and the "best available time" suggestion elsewhere in the app.

IF YOUR TEACHER ASKS...

"Is there a backend server?"

No — Firebase is the backend. Auth, database, and AI all run through Firebase's own services, authorized by Firestore Security Rules instead of server code. It's a real, common architecture pattern (Backend-as-a-Service), not a shortcut around building one.

"How do you stop someone faking a 5-star rating?"

Feedback is tied to one specific completed session between two specific real accounts, one rating per person per session (the document ID itself enforces that), and every stat shown is computed live from those real records — never edited or set directly.

"Why not just let AI calculate the whole match score?"

Explainability. A score a student acts on has to be traceable to real, fixed reasons — not a model's judgment call that could vary between runs or hallucinate a number.

"What happens if the AI fails or is slow?"

Every AI call has a deterministic fallback — a local parser or a template — so those features degrade gracefully instead of breaking. This isn't hypothetical: the free Gemini tier has a small quota and I hit it repeatedly while building, so the fallback path is genuinely tested, not just theoretical.

> **"Is this real data or a mock/demo?"**
>
> Real. Every account, rating, and session shown is an actual Firestore record created by going through the real registration/request/feedback flow — nothing in the UI is hardcoded sample data.
